

Modul 11 – CRUD menggunakan *Local storage*

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa mampu:

1. Memahami konsep *local storage* menggunakan emulator.
2. Memahami fungsi SharedPreferences.
3. Menyimpan, mengambil dan menghapus data dari *local storage*.
4. Menambah fitur CRUD data menggunakan SharedPreferences.

B. Dasar Teori

1. *Local storage*

Local storage adalah penyimpanan data pada perangkat pengguna secara lokal tanpa internet. Data yang disimpan di memori internal dan storage aplikasi.

2. CRUD *Local storage*

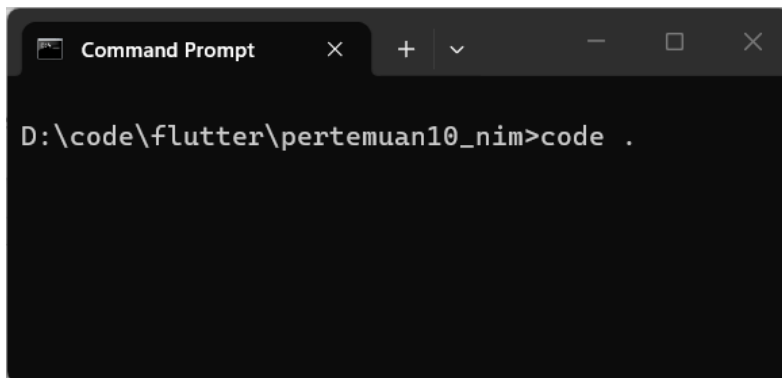
Menerapkan sistem CRUD pada flutter menggunakan local storage dengan emulator web (chrome) memiliki perbedaan dibandingkan menggunakan emulator android. Dikarenakan perbedaan port setiap menjalankan aplikasi maka data akan seperti hilang ketika aplikasi kembali dijalankan.

3. SharedPreferences

SharedPreferences adalah package Flutter untuk menyimpan data sederhana berbentuk: **String, Integer, Double, Boolean dan List<String>**. Data yang disimpan dalam bentuk key -> value.

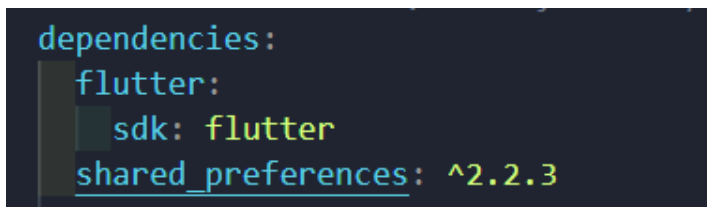
C. Praktikum

1. Silalahkan buka kembali project flutter **pertemuan10_nim** ke dalam VS Code.



```
D:\code\flutter\pertemuan10_nim>code .
```

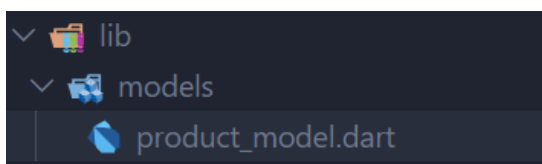
2. Buka file pubspec.yaml, pastikan dependency yang terinstall sama seperti berikut.



```
dependencies:  
  flutter:  
    sdk: flutter  
  shared_preferences: ^2.2.3
```

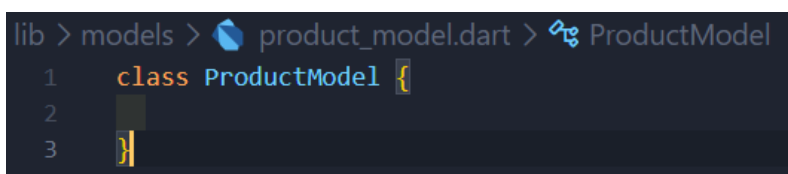
shared_preferences merupakan library/ package yang biasa digunakan dalam flutter untuk menyimpan data local di dalam aplikasi.

3. Jalankan project flutter tersebut lewat terminal dengan perintah **flutter run -d chrome --web-port=3000** dengan begitu port dari emulator chrome akan konsisten berjalan di port 3000.
4. Buat folder models di dalam lib kemudian tambahkan file product_model.dart



```
lib  
└── models  
    └── product_model.dart
```

5. Tambah class ProductModel didalamnya.



```
lib > models > product_model.dart > ProductModel  
1  class ProductModel {  
2  
3  }
```

Import package convert diatasnya

```
import 'dart:convert';
```

Inisialisasi variable data yang akan digunakan didalamnya

```
final String name;  
final String description;  
final int price;
```

Tambahkan constructor dibawahnya

```
ProductModel({  
  required this.name,  
  required this.description,  
  required this.price,  
});
```

Kemudian ubah object ke map, karena JSON bekerja dengan Map dan SharedPreferences tidak bisa simpan object langsung.

```
// OBJECT → MAP  
Map<String, dynamic> toMap() {  
  return {  
    'name': name,  
    'description': description,  
    'price': price,  
  };  
}
```

ubah dari map ke object menggunakan factory karena merupakan constructor khusus dan menghasilkan object baru.

```
// MAP → OBJECT  
factory ProductModel.fromMap(  
  Map<String, dynamic> map,  
) {  
  return ProductModel(  
    name: map['name'] ?? '',  
    description: map['description'] ?? '',  
    price: map['price'] ?? 0,  
  );  
}
```

Fungsi simbol ?? (Null Coalescing Operator) untuk mengantisipasi apabila data kosong.

Kemudian ubah object ke string untuk dilakukan encode karena SharedPreferences hanya bisa menyimpan String, setelah itu convert dari string ke object menggunakan factory untuk melakukan decode.

```
// OBJECT → JSON STRING
String toJson() => jsonEncode(toMap());

// JSON STRING → OBJECT
factory ProductModel.fromJson(String source) {
  return ProductModel.fromMap(
    jsonDecode(source),
  );
}
```

6. Buka file pages/home_page.dart dan tambahkan import post_model didalamnya.
7. Pada class state _HomePageState, inialisasi variable penampung daftar produk.

```
List<ProductModel> products = [];
```

Pada void initState() tambahkan method loadProducts()

```
@override
void initState() {
  super.initState();
  getUser();
  loadProducts();
}
```

8. Buat method loadProducts() dibawahnya

```
Future<void> loadProducts() async {
  final prefs = await SharedPreferences.getInstance();
  List<String> productList = prefs.getStringList('products') ?? [];
  setState(() {
    products = productList
      .map((item) => ProductModel.fromJson(item))
      .toList();
  });
}
```

Method diatas akan memuat data jika terdapat data produk maka akan ditampilkan terlebih dahulu.

9. Berikutnya buat method saveProduct()

```
Future<void> saveProducts() async {  
  final prefs = await SharedPreferences.getInstance();  
  List<String> productList = products.map((item) => item.toJson()).toList();  
  await prefs.setStringList('products', productList);  
}
```

Method ini berfungsi untuk menyimpan terhadap data produk yang dikelola.

10. Setelah itu tambahkan method untuk menambah produk

```
Future<void> addProduct(ProductModel product) async {  
  setState(() {  
    products.add(product);  
  });  
  await saveProducts();  
  ScaffoldMessenger.of(context).showSnackBar(  
    const SnackBar(content: Text("Produk berhasil ditambahkan")),  
  );  
}
```

Jika data berhasil ditambahkan maka akan disimpan melalui await saveProducts()

11. Buat juga method untuk editProduct()

```
Future<void> updateProduct(int index, ProductModel product) async {  
  setState(() {  
    products[index] = product;  
  });  
  await saveProducts();  
}
```

Sama dengan method addProduct() jika data sudah di perbaharui maka akan disimpan ke saveProduct()

12. Terakhir buat method untuk deleteProduct()

```
Future<void> deleteProduct(int index) async {  
  setState(() {  
    products.removeAt(index);  
  });  
  await saveProducts();  
  ScaffoldMessenger.of(  
    context,  
  ).showSnackBar(const SnackBar(content: Text("Produk berhasil dihapus")));  
}
```

13. Tambahkan method showForm()

```
void showForm({ProductModel? product, int? index}) {  
  TextEditingController nameController = TextEditingController(  
    text: product?.name ?? "",  
  );  
  TextEditingController descriptionController = TextEditingController(  
    text: product?.description ?? "",  
  );  
  TextEditingController priceController = TextEditingController(  
    text: product?.price.toString() ?? "",  
  ); ...  
}
```

Berikut dengan controller berdasarkan data produk didalamnya, method ini fungsinya untuk menampilkan formulir dialog untuk menambah dan mengedit data.

Tambahkan showDialog untuk menampung formulirnya

```
showDialog(  
  context: context,  
  builder: (_) => AlertDialog(  
    title: Text(product == null ? "Tambah Produk" : "Edit Produk"),  
    |  
  ), // AlertDialog  
);
```

Kemudian tambahkan property content

```
content: Column(  
  mainAxisAlignment: MainAxisAlignment.min,  
  children: [  
    TextField(  
      controller: nameController,  
      decoration: InputDecoration(labelText: "Nama"),  
    ), // TextField  
    TextField(  
      controller: descriptionController,  
      decoration: InputDecoration(labelText: "Deskripsi"),  
    ), // TextField  
    TextField(  
      controller: priceController,  
      decoration: InputDecoration(labelText: "Harga"),  
      keyboardType: TextInputType.number,  
    ), // TextField  
  ],  
), // Column
```

Tambahkan property action untuk menyimpan menampilkan tombol submit

```
actions: [
  ElevatedButton(
    onPressed: () {
      final newProduct = ProductModel(
        name: nameController.text,
        description: descriptionController.text,
        price: int.parse(priceController.text),
      ); // ProductModel
      if (product == null) {
        addProduct(newProduct);
      } else {
        updateProduct(index!, newProduct);
      }
      Navigator.pop(context);
    },
    child: Text("Simpan"),
  ), // ElevatedButton
],
```

14. Sekarang lihat pada bagian widget builder dan tambahkan kode berikut setelah Container profile.

```
Expanded(
  child: products.isEmpty
    ? const Center(child: Text("Belum ada produk"))
    : ListView.builder(
      itemCount: products.length,
      itemBuilder: (context, index) {
        final product = products[index];
      },
    ), // ListView.builder
), // Expanded
```

kode diatas berarti jika data produk kosong maka tampilkan pesan "Belum ada produk", tetapi jika ada maka tampilkan ke dalam widget ListView.builder() beserta widget Card dibawah ini

```

return Card(
  margin: const EdgeInsets.only(bottom: 12),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(15),
  ), // RoundedRectangleBorder
  child: ListTile(
    contentPadding: const EdgeInsets.all(15),
    title: Text(
      product.name,
      style: const TextStyle(
        fontWeight: FontWeight.bold,
      ), // TextStyle
    ), // Text
    subtitle: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const SizedBox(height: 5),
        Text("Rp ${product.price}"),
        const SizedBox(height: 5),
        Text(product.description),
      ],
    ), // Column
  ), // ListTile
); // Card

```

Setelah property subtitle tambah property leading untuk tombol edit

```

leading: IconButton(
  icon: const Icon(
    Icons.edit,
    color: Colors.orange,
  ), // Icon
  onPressed: () => showForm(product: products[index], index: index),
), // IconButton

```

Data tambahkan juga setelahnya property trailing untuk tombol hapus

```

trailing: IconButton(
  icon: const Icon(
    Icons.delete,
    color: Colors.red,
  ), // Icon
  onPressed: () => deleteProduct(index),
), // IconButton

```

15. Jika sudah silahkan save dan jalankan kembali aplikasi di emulator.